

REQ4 – Witch Mayhem of Potions

A. Potion Effect Implementation

Design 1: Abstract Potion Class with Subclasses and StatusEffect Objects

Pros	Cons
The potion itself (Potion) is separated from its effects (StatusEffect), ensuring distinct responsibilities and improving modularity.	Each New Effect Requires a StatusEffect Subclass: Creating a new effect (e.g., SpeedEffect) necessitates a new StatusEffect subclass, increasing the number of classes to manage.
Adding a new potion effect only requires a new Potion subclass and corresponding StatusEffect subclass, adhering to the Open-Closed Principle OCP.	
The same StatusEffect (e.g., PoisonEffect) can be triggered by different potions or items (if designed to allow this), promoting code reuse and DRY.	

Design 2: Hardcoded Effect Logic in Each Specific Potion Class

Pros	Cons
Direct for Simple Effects: For very simple effects (e.g., instant healing), hardcoding logic in the potion class is straightforward, reducing the need for additional classes.	Effect logic is bound to the potion class, limiting reuse in other contexts (e.g., non-potion items) and reducing modularity.
Minimal Initial Setup: Requires fewer classes and abstractions, allowing rapid prototyping for small systems with basic effects.	The potion class manages both identity and effect logic, mixing concerns and hindering maintainability.
	Similar effects (e.g. StrongHealingPotion) can lead to duplicated logic, violating DRY and increasing maintenance burden.

B. Potion Brewing System

Design 1: Modular Brewing via Pouch and Action Classes

Pros	Cons
Pouch manages recipe and material logic, while BrewAction executes brewing, ensuring modular and maintainable code.	A separate BrewAction is needed for each potion, increasing the number of classes and overall codebase complexity.
Defining recipes as Status lists enables easy modification and supports a dynamic, extensible crafting system.	
Shared material validation and consumption logic in Pouch reduces duplication and adheres to DRY principles.	
New potions can be added without modifying existing code, aligning with the Open/Closed Principle.	

Design 2: Player Class Directly Handles All Brewing Logic with Hardcoded Item Names

Pros	Cons
Consolidating brewing logic in Player can be simple and quick for small systems with few potions.	Brewing logic is tightly bound to Player and specific items, limiting modularity and flexibility.
Allows rapid implementation during early development with basic brewing needs.	Adding new recipes requires modifying complex if-else structures, increasing error risk and maintenance overhead
	Material checks and consumption are repeated across branches, violating DRY principles.

C. Item Interaction (e.g., Collecting Water/Creating Ponds)

Design 1: Player Class Checks Environment and Inventory to Determine Actions

Pros	Cons
Reduces the need for dedicated Action classes by handling logic directly in Player, simplifying the codebase for small systems.	Logic for determining available actions is scattered across Player, reducing clarity and maintainability.
Allows quick implementation of interactions by adding checks in playTurn, suitable for early development or minimal systems.	

Design 2: Ground and Item Provide Specific Actions via allowableActions

Pros	Cons
Context-specific interactions (e.g. CollectWaterAction for Pond) ensures intuitive and relevant gameplay.	Minor boilerplates across similar Action classes may require abstraction to reduce redundancy.
Encapsulated logic within each Action class enhances maintainability and clarity.	
Highly extensible—new interactions (e.g. CreateLakeAction) can be added without modifying existing code, supporting the Open/Closed Principle.	

The diagram above shows the final design of requirement 4, which utilizes Design 2 for **Potion Effect** and **Brewing System implementation**, Design 1 for **Item Interaction Implementation**.

There is a must to implement the Meat Abstract class here as we want to reduce the code duplication as we assume for now all meat can be eaten and their logic after eating remains the same. Furthermore, different meats have different status e.g. CURSED and BLESSED which means we cannot have a single meat class to handle everything and for this requirement our design assumes that every meat class has a constant status no same type of meat can have the same status, which means passing status enumeration as arguments is not suitable.

This final design adheres strongly to **Single Responsibility Principle (SRP)**. Each `StatusEffect` subclass focuses solely on effect logic, while `Potion` subclasses handle potion-specific data (e.g., name, description). The `Pouch` class is responsible only for material and recipe management; each `Brew Action` class is responsible solely for executing the brewing step. Item-interaction logic resides in discrete `Action` classes, leaving the `Player` class free from environment specific checks.

The **Open/Closed Principle OCP** is supported because new potions and effects can be added simply by writing new `Potion` and `StatusEffect` subclasses, with no changes to existing code. Similarly, adding new brewing recipes only requires creating a new `Brew Action` and registering it with the `Pouch`, without touching existing classes. New ground or item interactions (e.g., `CreateLakeAction`) can be introduced by defining a new `Action` class and updating the corresponding item or ground type's `allowableActions` with no existing code is modified.

Liskov Substitution Principle (LSP) is respected: any subclass of `StatusEffect` may be used wherever a `StatusEffect` is expected (e.g., `PoisonEffect` vs. `SpeedEffect`), and any new `BrewAction` can substitute in the same context as other brewing actions. Item-interaction `Action` subclasses can be interchanged freely when provided via `allowableActions`, without altering consumer code.

The **Interface Segregation Principle (ISP)** guides the design by keeping interfaces small. The `StatusEffect` interface defines only methods like `tick()` and `toString()` as referenced in the engine class. For the brewing system, `Pouch` exposes only material-checking and consumption methods; `BrewAction` implements only what is necessary for crafting, without forcing clients to depend on unrelated methods.

Through **Dependency Inversion Principle (DIP)**, high-level modules depend on abstractions. For instance, `Player` depends on `Action` abstractions rather than concrete implementations to perform item interactions. The brewing system depends on `BrewAction` abstractions and `Pouch` abstractions, not on specific recipes. The potion system depends on a generic `StatusEffect` interface, not concrete effect classes.

The design follows **DRY (Don't Repeat Yourself)** by centralizing effect logic in dedicated `StatusEffect` classes. If two potions share the same effect, both simply reference the same `StatusEffect` subclass. In the brewing system, material checking logic is implemented once in `Pouch` and reused by all `BrewAction` classes. Item-interaction logic is encapsulated in individual `Action` classes (e.g., `CollectWaterAction`), so common functionality (e.g., verifying a container is empty) can be shared via helper methods instead of repeated.

In this design, the dominant forms of **connascence** are **Connascence of Name (CoN)** and **Connascence of Type (CoT)**. For instance, `Potion` subclasses and `StatusEffect` subclasses must agree on method names like `tick()` and consistent type usage to interact properly. These

connections are manageable and well-contained due to interface-driven design. Importantly, the system avoids tighter forms like **Connascence of Position (CoP)** or **Connascence of Meaning (CoM)** by encapsulating logic in clearly named and well-isolated classes. This results in a modular, extensible structure where responsibilities are well-separated, minimizing ripple effects during change and supporting future evolution without introducing unnecessary fragility.

The system is designed with future maintainability in mind, allowing for straightforward extensibility without altering existing code. For instance, adding a new potion effect simply involves creating a new `StatusEffect` subclass (such as `RegenerationEffect`) and its corresponding `Potion` subclass. Similarly, introducing new brewing recipes only requires defining a new `BrewAction` class (e.g., `BrewInvisibilityPotionAction`) and registering its recipe within the `Pouch`, without the need to modify existing actions or classes. Furthermore, extending item interactions such as implementing a “`FillWellAction`” for a new well-type ground can be achieved by creating the action class and overriding the `allowableActions()` method in the specific ground type, leaving the core `Player` logic unchanged. Furthermore, the presence of abstract meat class

The system ensures a clear separation of concerns and demonstrates strong adherence to SOLID principles, establishing a robust foundation for organic growth. Its modular architecture promotes clean, maintainable code by isolating responsibilities and minimizing dependencies. As a result, new features whether potions, brewing recipes, or item interactions can be introduced seamlessly without invasive refactoring, enabling the system to evolve gracefully over time.